

Why Should You Learn JavaScript? (IMP Qns)

There are many good reasons to learn JavaScript, especially if you're interested in websites, apps, or programming in general.

1. It's the Language of the Web

JavaScript is the main language that web browsers understand. That means whenever you visit a website and see buttons, forms, sliders, or animations—JavaScript is making them work.

2. You Already Have What You Need

If you have a web browser and a simple text editor, you can start coding JavaScript right away. No special setup is needed to begin.

3. Great for Beginners

JavaScript is beginner-friendly. You can build small apps and interactive web pages even after learning just the basics. You'll be surprised how quickly you can make something cool.

4. You Can Build Many Kinds of Projects

JavaScript isn't just for web pages. You can also use it to:

- Build games
- Create mobile and desktop apps
- Write code for the backend (server)
- Automate tasks

5. Supports Many Programming Styles

As you learn more, you'll discover different ways to write JavaScript—like object-oriented, functional, or procedural programming. Don't worry about those words now; just know that JavaScript is flexible.

6. Lots of Tools and Frameworks

Once you learn the basics, there are powerful tools that help you build even faster, like:

- React
- Vue
- Angular
- Node.js

These tools make it easier to build modern websites and apps.

7. Big Helpful Community

JavaScript is one of the most popular programming languages in the world. If you ever get stuck, there are forums, tutorials, and helpful people online (like on Stack Overflow) to guide you.

8. Easy to Set Up

You can start learning JavaScript with just your browser and a code editor like VS Code. These tools even help by coloring your code, showing errors, and suggesting fixes.

How Does the Browser Understand JavaScript?

JavaScript is an Interpreted Language

- JavaScript is not compiled like some other languages (e.g., C or Java).
- It's interpreted, which means the browser reads and runs the code line by line at runtime.
- There's no need to convert it into machine code before running. The browser does that automatically using something called a JavaScript engine.

JavaScript Engine (Interpreter)

Each modern browser has a **built-in JavaScript engine** (also called an interpreter). Examples include:

Browser	JavaScript Engine
Chrome	V8
Firefox	SpiderMonkey
Safari	JavaScriptCore
Edge	Chakra (old), now also uses V8

When you open a web page, the browser finds the JavaScript in it, and the engine **reads, understands, and executes** it on the fly.

Web Pages Are Made of 3 Main Parts

When you visit a website, you're not just loading JavaScript. You're loading a mix of **three core technologies**:

1. **HTML (HyperText Markup Language)** – This gives **structure** to the web page.

- It decides **what is shown** on the page: headings, paragraphs, images, buttons, etc.
- Example:

```
<html>
```

```
<body>
```

```
Hello world!
```

```
</body>
```

```
</html>
```

2. **CSS (Cascading Style Sheets)** – This defines the **style and layout** of the page.

- It handles **colors, fonts, positioning, background, spacing**, etc.
- For example, turning text blue or making a heading centered.

3. **JavaScript** – This adds **functionality** and **interaction**.

- It can respond to user actions (like clicks or typing), update the page without reloading, and even send data to/from a server.
- JavaScript brings the page to life.

How JavaScript Works in the Browser

1. The browser **downloads** the HTML, CSS, and JavaScript files from the website.
2. It **renders** the HTML first (displays content).
3. CSS is applied to **style** the elements.
4. The JavaScript engine **interprets and executes** any JavaScript found in the page (either inside <script> tags or from external files).
5. JavaScript can then:
 - Access and change the content (using the **DOM** – Document Object Model).
 - React to user actions (like clicks, typing, scrolling).
 - Send/receive data from servers.

What is ECMAScript? (Imp)

You might hear the term **ECMAScript (ES)** when learning JavaScript. Here's what it means:

- ECMAScript is the **official standard** or **rulebook** for the JavaScript language.
- JavaScript follows these rules to make sure it works consistently across browsers.
- **ES6** (ECMAScript 6) is a popular version with many powerful features like:
 - let, const
 - Arrow functions (=>)

- Classes
- Modules
- Template literals (Hello \${name})

So, when browser makers build or update their JavaScript engines, they follow the **ECMAScript standard** to ensure compatibility.

Using the Browser Console

Modern web browsers come with **developer tools** that let you **see and test the code** running behind any web page. One very important part of these tools is the **Console**—a place where you can write and run JavaScript instantly.

How to Open the Developer Tools Console

Depending on your computer and browser, there are a few ways to open the **console**:

On Windows:

- Press **F12**
OR
- Right-click on the page and select "**Inspect**", then click the "**Console**" tab.

On macOS:

- Press **Command + Option + I**
OR
- Right-click and choose "**Inspect**", then go to the "**Console**" tab.

What You See

Once the developer tools open, you'll see several tabs:

- **Elements** – shows the HTML of the page.
- **Console** – where you can **write and test JavaScript code**.
- **Network, Sources, Application, etc.** – advanced tools (don't worry about them for now).

Click on the **Console** tab to open it.

What Can You Do in the Console?

The Console lets you:

- Run **JavaScript code** instantly.
- See **errors or messages** from the website's JavaScript.
- Debug your code using `console.log()`.

Example:

Type the following in the Console and press **Enter**:

```
console.log("Hello from the console!");
```

You'll see the message printed below. This is helpful for:

- Testing small code snippets
- Checking variable values
- Understanding how your scripts work

Why is the Console Useful?

- You can test ideas without writing full HTML files.
- You can check if your JavaScript code works.
- You can explore how websites behave.
- You can practice as you learn, without needing to set up a full project.

Tips

- Use `console.log("message")` to print messages.
- Use arrow up/down to scroll through previous commands.
- The browser console **only runs JavaScript**. You can't run HTML or CSS directly in it.

Example-01:

```
<body>  
  <script>  
    // This is a simple JavaScript code snippet  
    console.log ("Hello, Javascript!");  
  </script>  
</body>
```


Example-02:

<script>

a=10;

b=20;

c=a+b;

console.log(a);

console.log(b);

console.log(c);

c=a-b;

console.log(c);

c=a*b;

console.log(c);

c=a/b;

console.log(c);

c=a%b;

console.log(c);

c=a**b;

console.log(c);

c=a++;

console.log(c);

c=++a;

console.log(c);

c=a--;

console.log(c);

```
c=--a;  
console.log(c);  
c=a+=b;  
console.log(c);  
c=a-=b;  
console.log(c);  
c=a*=b;  
console.log(c);  
c=a/=b;  
console.log(c);  
c=a%=b;  
console.log(c);  
c=a**=b;  
console.log(c);  
</script>
```

Example-03:

```
<script>
    a=10;
    console.log(a);
    a=50;
    console.log(a);
    b=20;
    a=b;
    console.log(a);
</script>
```

Adding JavaScript to a Web Page

JavaScript adds interactivity and functionality to your web pages. But first, you need to **link or include JavaScript in your HTML file**. There are two common ways to do this:

1. Writing JavaScript Directly in the HTML File

You can write JavaScript **inside the <script> tag** directly in the HTML file.

Basic Example:

```
<!DOCTYPE html>
<html>
  <head>
    <title>My First JS Page</title>
  </head>
```

```
<body>
  <script type="text/javascript">
    alert ("Hi there!");
  </script>
</body>
</html>
```

What this does:

- When the page is opened in the browser, you'll get a **pop-up alert** saying **"Hi there!"**.
- The JavaScript code (alert ("Hi there!")) is inside the <script> tag.

Note:

- You can place this script inside the <head> or <body>, but placing it **at the end of the body** is better for performance.

2. Linking an External JavaScript File

Instead of writing JavaScript directly in your HTML, you can put the code in a **separate .js file** and **link it using the <script> tag**.

Folder Structure:

HTML File (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>External JavaScript Example</title>
```

```
</head>
<body>
  <h1>Welcome to My Page</h1>
  <script src="script.js"></script>
</body>
</html>
```

JavaScript File (script.js)

```
alert ("Hi from script.js!");
```

Example-01:

```
<script>
  alert ("hello Mca");
</script>
```

Prompt

Another thing we would like to show you here is also a command prompt. It works very much like an alert, but instead, it takes input from the user. We will learn how to store variables very soon, and once you know that, you can store the result of this prompt function and do something with it. Go ahead and change the alert () to a prompt () in the Hi.html file, for example, like this:

```
prompt("Hi! How are you?");
```

Then, go ahead and refresh the HTML. You will get a popup with an input box in which you can enter text.

Page prompting for use input The value you (or any other user) enter will be returned to the script, and can be used in your code! This is great for getting user input to shape the way your code works.

Example:

```
<script>  
    a=prompt("Enter a value:");  
    console.log(a);  
    document.write("You entered: " + a);  
</script>
```

Generating Random Numbers in JavaScript

JavaScript provides a **built-in method** for generating random numbers using the Math object.

Syntax

```
Math.random();
```

What Does It Do?

- Math.random() generates a **pseudo-random** number between 0 (inclusive) and 1 (exclusive).
- That means the output will be a **floating-point number** like 0.23748, 0.76823, etc.

- It **never** returns 1, but can return a number very close to 1 (e.g., 0.999999).

Example 1: Basic Random Number

```
console.log(Math.random());
```

Output: A random decimal between 0 and 1

Example Output: 0.573829114

Custom Range – Random Number Between 0 and 100

If you want a number **between 0 and 100**, multiply the result by 100:

```
console.log(Math.random() * 100);
```

Output: A random **decimal** between 0 and 100

Example Output: 73.4928342

Round It to a Whole Number – Use Math.floor()

If you want an **integer** between 0 and 99, you can **round down** the decimal using Math.floor():

```
console.log(Math.floor(Math.random() * 100));
```

Output: An **integer** between 0 and 99

Example Output: 42

Explanation of Each Part:

Expression

Meaning

`Math.random()`

Generates a decimal between 0 and 1

`Math.random() *
100`

Scales the range to 0–100

`Math.floor(...)`

Rounds the decimal **down** to the nearest integer

Note:

- This is just a **simple way** to get random values for games, exercises, quizzes, etc.
- These examples are often used in **early chapters** of learning JavaScript just for fun and practice.
- You don't need to understand the full internal working of `Math.random()` right now.
- Later, in **Chapter 8: Built-In JavaScript Methods**, you'll dive deeper into how it all works.

Try It Yourself:

```
for (let i = 0; i < 5; i++) {  
  console.log (Math.floor(Math.random() * 100));  
}
```


This will print 5 random integers between 0 and 99.

Tip:

If you want a **random number between min and max (inclusive)**:

```
function getRandomInt (min, max) {  
    return Math.floor(Math.random() * (max - min + 1)) + min;  
}
```

Example:

```
console.log (getRandomInt (10, 20)); // Random number between 10  
and 20
```

Variables in JavaScript

What Are Variables?

- **Variables** are like containers used to **store data**.
- They let you **label, store, and manipulate** information in your programs.
- Variables are called "variables" because their value can **vary** or **change** as the code runs.

Example:

```
let firstname = "Maaike";
```

```
let x = 2;
```

```
firstname = "Edward";
```

```
x = 7;
```

In the above example:

- "Maaike" is **replaced** with "Edward".
- 2 becomes 7.

Why Are Variables Important?

Without variables:

- Code would do **exactly the same thing** every time.
- Variables make your programs **dynamic**, adaptable, and reusable.

Declaring Variables

To **declare** a variable means to **create it for the first time**.

Keywords used to declare variables:

Keyword	Can be Reassigned	Scope Type	Notes
let	Yes	Block	Recommended
var	Yes	Global or Function	Not recommended (old style)
const	No	Block	Used for constants

Example:

```
let firstname = "Maria";
```

```
firstname = "Jacky"; // Reassignment allowed
```

```
const PI = 3.14159;
```

```
// PI = 3.14; Error: Assignment to constant variable
```

```
var age = 25;
```

Understanding let, var, and const

let

- Introduced in ES6.
- Used for **variables that can change**.
- Has **block-level scope**, meaning it only exists within { }.

```
let a = 10;
```

```
a = 20; // Works
```

var

- Older way to declare variables.
- Has **function/global scope**, meaning it can **leak out** of blocks.
- **Avoid using var** in modern JavaScript due to scope confusion.

```
var b = 15;
```

```
b = 25; // Works
```

const

- Used to declare **constants** (unchangeable values).
- Must be **assigned at the time of declaration**.
- Cannot be reassigned.
- Also block-scoped like let.

```
const pi = 3.14159;
```

```
// pi = 3.14; Error
```

Scope – Where Variables Live

Scope Type	Description
Global Scope	Variables are accessible everywhere in your script (mostly with var).
Block Scope	Variables exist only within the block { } in which they are declared (with let and const).

USBM

Block Example:

```
{  
  let name = "Alex";  
  console.log(name); // Accessible here  
}  
  
console.log(name); // Error: name is not defined
```

Naming Variables: Best Practices

1. Start with a lowercase letter

- ✓ firstname, userAge
- ✗ Firstname, X

2. Be descriptive

- ✓ totalMarks, isLoggedIn
- ✗ x, y, temp (unless in temporary or math context)

3. No spaces

- ✗ first name
- ✓ first_name, firstName (camelCase is preferred)

4. Cannot start with a number

- ✗ 1value
- ✓ value1

Hardcoded vs Dynamic Variables

- In early examples, variables are **hardcoded**:

```
let username = "admin"; // Hardcoded
```
- In real-world apps, variable values often come from:
 - User inputs
 - API calls
 - Databases
 - Other program parts

Dynamic values make code flexible and interactive.

Summary Table

Term	Description
let	Changeable, block-scoped variable
var	Changeable, function/global scoped variable (avoid using)
const	Constant value, cannot be reassigned
Hardcoded	Value set directly in the script
Dynamic value	Value comes from external input or calculation
Scope	The area in which a variable can be accessed

Tip

You'll be using let and const the most in modern JavaScript. Start with const by default, and switch to let **only when** you need to change the value.